Machine Learning (934G5) Coursework

Candidate Number: 291723

Word count: XXXX

1 Features & Labels

The following variable names constitute the **labels** used in this multilayer perceptron (MLP) model, corresponding to the values 7 days ahead in time of date of the current data instance:

- `air_pressure_mean_plus7days`
- `rain_total_plus7days`
- `wind_direction_sin_mean_plus7days`
- `wind_direction_cos_mean_plus7days`
- `wind_speed_mean_plus7days`

The **features** used in this MLP model are listed below. Each variable has lagged variants, which correspond to an n-shifted version of the variable corresponding to the number of days n prior to the present day of the data instance (for example, `air_pressure_mean_lag1` refers to the value `air_pressure_mean` took on the previous day, relative to a given data instance's current date):

- `air_pressure_mean` (lag6 to current)
- `rain_total` (lag6 to current)
- `wind_direction_sin_mean` (lag6 to current)
- `wind_direction_cos_mean` (lag6 to current)
- `wind_speed_mean` (lag6 to current)

See Section 2 for justification.

1.1 Feature Dimensions

To train the present MLP, a train-validation-test paradigm is used. Thus dimensionalities are reported for each stage. Given that the aim is to provide week-ahead forecasts for 4 weather variables, the present MLP will complete a regression task, making a real-valued prediction for each output variable. Thus, the labels (i.e. output vectors $\mathbf{y_i}$) will have a the following dimensionalities:

$$\mathbf{y}_{\text{train}} \in \mathbb{R}^{963,806\times 5}, \quad \mathbf{y}_{\text{validate}} \in \mathbb{R}^{209,401\times 5}, \quad \mathbf{y}_{\text{test}} \in \mathbb{R}^{141,986\times 5}$$

The features (i.e. input vectors $\mathbf{x_i}$) include each weather variable for the current date of the data instance and its lagged variants, and so will have a dimensionalities of:

$$\mathbf{x}_{\text{train}} \in \mathbb{R}^{963,806\times 39}, \quad \mathbf{x}_{\text{validate}} \in \mathbb{R}^{209,401\times 39}, \quad \mathbf{x}_{\text{test}} \in \mathbb{R}^{141,986\times 39}$$

2 Preprocessing

The raw data files provide monthly CSV files for each feature. The first preprocessing step was to combine these CSV files into a single dataframe for each feature. Features in each dataframe are recorded at a 3-hourly resolution. Thus, the arithmetic mean of these 3-hourly columns for each day at each location was found for the `air pressure`, `wind direction` and `wind speed` features, while the `rain` feature was multiplied by 1×10^3 to represent rain fall in millilitres and summed to provide a total rain fall for each day at each location.

Furthermore, prior to taking the mean of `wind direction` 3-hourly values, this variable was converted to radians and then its sine and cosine components were extracted. This provides two variables for `wind direction`, namely `wind_direction_sin` and `wind_direction_cos`. Such a preprocessing step was used as `wind direction` is circular, and so values which are numerically distant (e.g. -175 and 175) actually refer to directions which are close in space. Using a sine and cosine representation allows this closeness to be captured whereby the sine component represents the east-to-west lateral direction and the cosine components represents the north-to-south vertical direction.

Similarly, the day of the year can be represented by sine and cosine components as well since this is also a circular variable. Using the numeric `year`, `month` and `day` variables, a `date` variable was constructed and a `day_of_year` variable derived from that. This `day_of_year` variable is then converted to `day_sin` and `day_cos` variables, whereby Dec 31st and Jan 1st will have very similar sine and cosine representations, reflecting the fact that they are consecutive days rather than maximally distant as with a linear representation of the date.

To obtain the weather variable values a week ahead, a new variable was created for each weather variable which constitutes a 7-day shifted version of that variable (denoted with the suffix `_plus7days`). Moreover, to obtain weather values for previous days, lagged variants of each weather variable was created for the previous 13 days, providing up to 2 weeks of context to use as prediction for the weather values a week ahead. These lagged variants are denoted with the suffix `_lagX`, where `X` is the number of days prior to the current date of the data instance.

The dataset was then split into train, validate and test subsets (feature set + label set). The train set refers to data instances occurring in 2023 (starting at 2023-01-15) and 2024, the validate set refers to data instances from 1st Jan 2025 to 30th April 2025, and the test set refers to data instances occurring from 1st May 2025 to 2025-08-23. This chronological partitioning was chosen over randomised subsets as in a real-world context, a predictive model would not have access to future time points and would not aim to predict time points which have already occurred. A random subset approach would introduce data leakage, whereby the model would have access to future time points during training that it could then use to "predict" previous time points. In the real world, we do not have access to future time points (hence the need for predictive modelling) and so the training of this MLP should reflect this constraint.

Lastly, these train/validate/test subsets were standardised. Standardisation is a type of data transformation which takes each data point x in a variable, subtracts that variable's mean

μ and divides this by the standard deviation of the variable σ. This produces a z-score for each data point, where the transformed data now has a mean of 0 and a standard deviation of 1:

$$z = \frac{x - \mu}{\sigma}$$

Standardisation helps to avoid issues in gradient decsent with variables varying in scale. Variables on a larger scale (e.g. air pressure $\approx 10^5$) than others (e.g. wind speed $\approx 10^1$) will have weights of larger magnitudes. Gradient descent will focus on these larger magnitude weights more intensively than smaller weights and thus will "optimise" on the larger magnitude variables moreso than smaller magnitude variables. We instead want the MLP to learn from the variation in the data, not the variation in magitudes between variables, hence the use of standardisation.

3 Model

The MLP used in this report is illustrated in **Figure 1**, using the same feature dimensions shown in Section 1.1.

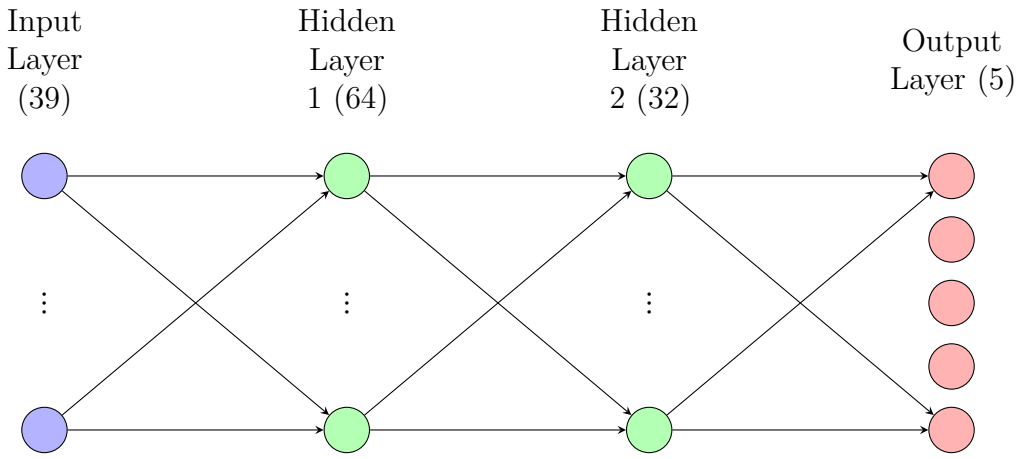


Figure 1. The Multilayer Perceptron Architecture Used in this Report.

A multiperceptron class was constructed using PyTorch, using 2 hidden layers, each with a rectified linear unit (ReLU) activation function. ReLU is well-suited as it helps reduce vanishing gradients during backpropagation. With backpropagation, we find the loss with respect to the weights of layer L via the chain rule:

$$\frac{\partial L}{\partial w^{(L)}} = \frac{\partial L}{\partial a^{(L)}} \frac{\partial a^{(L)}}{\partial z^{(L)}} \frac{\partial z^{(L)}}{\partial w^{(L)}}$$

When the gradient passes backwards through the network during backpropagation, the $\frac{\partial a^{(L)}}{\partial z^{(L)}}$ term is the change in activation of layer L due to a change in the input to the activation

of layer L from the linear models in the neurons of layer L. With ReLU, this derivative is simply 1 when the input is positive, and 0 otherwise. Thus, the ReLU activation function either simply passes the gradient backwards without applying any scaling (when equal to 1), or blocks it entirely (when equal to 0). Alternative activation functions like the hyperbolic tangent or sigmoid functions often produce gradients which are close to 0 and thus cause the gradient to shrink as it passes back through the network. This prevents earlier layers from being updated as much as latter layers, leading to vanishing gradients. With multiple layers in this MLP, ReLU is an appropriate activation function.

The train and validation sets are primarily used to assess overfitting. For instance, if validation losses begin to rise rather than fall with training losses, we have a clear indication that the model is increasingly overfitting to the data during training and thus failing to generalise to unseen data. This MLP is thus trained over multiple epochs (full passes over the training data). Within each epoch, the train data is randomly permuted to reduce the impact of sequential data points being highly correlated (i.e. autocorrelation - the previous data point may correspond to yesterday's weather, which will correlate strongly with today's). We then loop over batches from this permutation to train the model. After each epoch, the overall training and validation losses are recorded to allow observations to be made regarding model fit. Finally, the model is then tested on the test subset and evaluated in terms of its loss. Generally speaking, it is desirable for both losses to be as small as possible, but more importantly for the distance between the training and valdiation loss to converge to be as small as possible. This situation indicates that the model has converged on weights which generalise well to unseen data.

This MLP model uses the following hyperparaters, shown in **Table 1**:

Table 1. MLP Model Hyperparameters

parameter	value
Batch Size	512
Dropout	0.2
Epochs	10
Hidden Layer Sizes	[64, 32]
Input Size	39
Learning Rate	1e-05
Number of Lagged Days Used	6
Output Size	5
Test Sample Size	141986
Train Sample Size	963806
Validate Sample Size	209401
Weight Decay	1e-05

The batch size used (512) determines how many rows of training data are used at each iteration of an epoch. A smaller batch size increases computation time but can improve generalisability as the model has fewer examples to update its weights from at each iteration,

discouraging overfitting. However, a larger batch size will provide faster convergence, so a trade-off is to be made.

The dropout rate (0.2) determines the probability that a neuron is deactivated during each training iteration. This mechanism helps reduce overfitting because the network architecture effectively changes at each forward pass, preventing neurons from becoming overly dependent on specific, consistent pathways. This means the network behaves like an ensemble of subnetworks which are less sensitive to noise in the data. If the dropout rate is too high the network may fail to learn patterns in the data as effectively (underfitting), but if its too low the benefits for reducing overfitting are not gained.

The number of epochs (10) determines how many times the MLP passes over the training data. Having multiple epochs provides more opportunity for the model to converge on an optimal estimate for its weights. The number of epochs is chosen by evaluating the training and validation loss after each epoch. However, if losses fail to change significantly, additional epochs of training may introduce overfitting.

The number of hidden layers (2) determines the different levels of abstraction the model can learn about in the data. Generally, earlier layers will learn simpler patterns in the data, whilst latter layers learn more abstract relationships. A deeper network is useful for tasks it makes sense to evaluate data at different levels of abstraction (i.e. image classification), but for a regression task with no clear hierarchical relationships in the data, this does not apply as much. For instance, wind speed and rain fall don't really come together to form anything more abstract that simply "weather", whereas a picture of a dog has smaller components (pixels) which form larger components (ears, tail, nose) which then form more abstract objects (a dog). As such, it makes sense to keep the number of hidden layers relatively small for this task.

Regarding the number of nodes in each layer ([64, 32]), this determines the model's ability to capture nonlinear relationships. It makes more sense to increase the size of hidden layers in a regression task as we can learn more complex relationships between our features. That being said, too many nodes will result in overfitting as we are learning too many relationships, some which may just be noise unique to our training set and not present in unseen data. To avoid an overly complex model, keeping the size of hidden layers small, and only increasing if it results in a meaningful performance gain, is desirable.

The learning rate (0.00001) determines the extent to which weights are updated due to errors in predictions. This learning rate is multiplied with the calculated gradient (change in loss over change in weights) to downscale it. The aim with gradient descent is to find the global minimum (or more realistically an optimal local minimum) of the loss function with respect to the weights of the model.

A learning rate which is too high results in a model which trusts the data too much and overshoots its adjustments, potentially leading to the loss diverging from minima. Too small and the model may fail to learn from the data much at all and thus not converge to any minima within a reasonable amount of time. Typical learning rates for MLPs are on the order of 10^{-3} to 10^{-5}, and a reasonable value for the model/data can be inferred from observing its effect on training and validation loss.

Weight decay refers to a technique used to regularised model weights. This penalises the model by adding an extra term to the loss at each iteration. If the model assigns high weights, this term becomes large which inflates the loss, something the model is trying to minimise. This prevents overfitting as it encourages smaller and smoothed weights, rather than weights which are larger and highly adapted to the training data.

4 Generalisability

The present MLP used 963,806 data instances for training, 209,401 data instances for validation and 141,986 data instances for testing. **Figure 2** shows the training and validation losses over each epoch of the training process. The loss function used for training was the mean squared error (MSE):

$$\text{MSE} = \frac{1}{N}\sum_{i=1}^{N}(y_i - \hat{y}_i)^2$$

Since MSE squares the residuals (i.e. the difference between the predicted and true values - $y_i - \hat{y}_i$), larger residuals will make a greater contribution to the overall error than smaller residuals. Thus MSE is useful for punishing large errors made by the MLP model. Given that in this project we are creating a "multilayer perceptron model that forecasts four hurricane-relevant measures (rain precipitation rate, wind speed, wind direction, surface air pressure)", our interest is in more extreme weather events (i.e. hurricanes). Thus, our model provides more utility for this task if it is more optimised for predicting rare weather events, rather than optimising for typical weather. MSE's focus on larger errors therefore aids in this research objective.

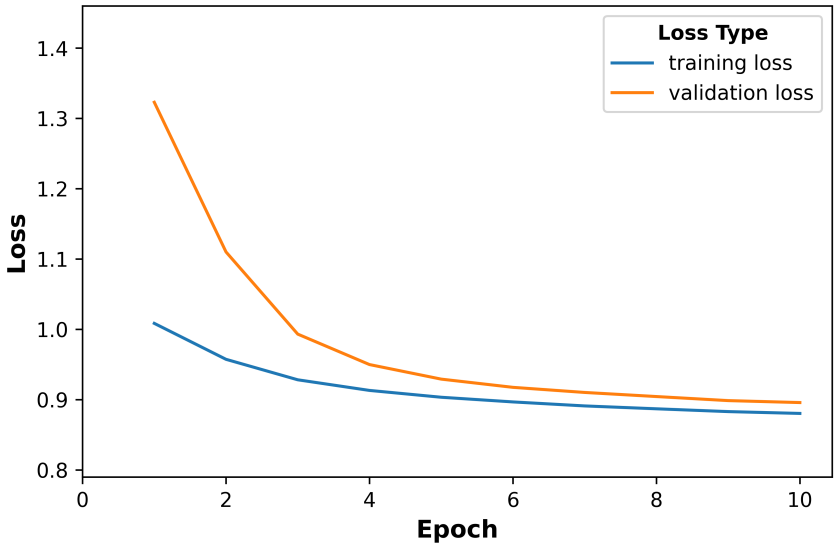


Figure 2. Training and Validation Loss over Epochs

As seen in **Figure 2**, training loss falls gradually over the 10. Importantly, validation loss, whilst initially higher, quickly converges to being slightly higher than the training loss. This plot therefore provides evidence that our model has good generalisation to unseen data. That being said, generalisation alone is not enough, we also want our predictions to be accurate.

Table 2 shows the results from the test set

Table 2. MLP Test Results

Label	RMSE	Mean	Std. Deviation	RMSE / Std. Dev
air_pressure	601.285	100951.127	1079.649	0.557
rain	0.479	0.12	0.413	1.16
wind_direction_sin	0.617	0.015	0.638	0.967
wind_direction_cos	0.593	-0.023	0.647	0.917
wind_speed	1.491	3.735	1.854	0.804